##SEPM-001

```
import java.util.concurrent.atomic.AtomicLong;
public class Saiba_Thread {
  static AtomicLong safeStatic = new AtomicLong(0);
  static long unsafeStatic = 0;
  static class CounterTask extends Thread {
    long instanceCount = 0;
    long increments;
    boolean threadsafe;
  CounterTask(long increments, boolean threadsafe) {
    this.increments = increments;
    this.threadsafe = threadsafe;
    }
  public void run() {
   for(long i=0; i<increments, i++) {
     instanceCount++;
    if (threadsafe) safeStatic.incrementAndGet();
    else
     unsafeStatic++;
      }
    }
  }
```

IT-24006

CamScanner

```
public static void main (String [ ] args)
        throws InterruptedException {
    int n = Integer.parseInt (args [0]);
    long m = long.parseLong (args [1]);
    boolean threadSafe = Boolean.parseBoolean(args[2]);

CounterTask [ ] tasks = new CounterTask [n];
for (int i = 0; i<n, i++) {
  tasks [i] = new CounterTask (m, threadSafe);
    tasks [i].start ();
      }
for (CounterTask t : tasks)
    t.join();
long nonstatic =0;
for (CounterTask t : tasks)
  nonStatic += t.instanceCount;
long staticCount = threadSafe? safeStatic.get(); unsafeStatic
long diff = Math.abs (staticCount - nonStatic);
double pct = (nonstatic ==0)? (staticCount==0?
                0.0 : Double.NaN): diff * 100.0/
                    nonstatic;
```

```
System.out.println("Threads: "+n+", Increment/
    thread: "+m+", Thread-safe: "+threadSafe);
System.out.println("Expected: "+(long) n*m);
System.out.println("Static count: "+ staticCount);
System.out.println("Non-static total: "+nonStatic);
System.out.println("Absolute difference: "+diff);
System.out.println("Percentage difference: %.4f%%",
                                        pct);
    }
}
```